

C# / .NET Learning Roadmap

Topic Notes — Section 6: Delegates, Events, and Functional Constructs

Topics: 6.1 through 6.4

Date: 29 June 2026

Section 6 — Delegates, Events, and Functional Constructs

This section covers all four roadmap topics: 6.1 (Delegates — Func, Action, Predicate), 6.2 (Multicast delegates), 6.3 (Events — event keyword, publisher/subscriber), and 6.4 (Lambda expressions).

Note on order: Lambda expressions (6.4) are taught first because delegates and events use them constantly. Understanding lambdas makes everything else in this section click immediately.

6.4. Lambda Expressions (taught first — foundation for 6.1–6.3)

Builds on: 1.11 (methods — lambdas are anonymous methods), 1.7 (control flow — lambdas contain expressions and statements), 5.2 (Where, Select — you have been writing lambdas throughout LINQ).

What are lambda expressions and why do they exist?

A lambda is a concise way to write an anonymous function — a function without a name, defined inline at the point where it is used. Before lambdas, passing behaviour required either a named method or a verbose anonymous method:

```
// Named method -- verbose, defined elsewhere:
private bool IsPending(Order o) { return o.Status == "Pending"; }
orders.FindAll(IsPending);

// Anonymous method (C# 2) -- still verbose:
orders.FindAll(delegate(Order o) { return o.Status == "Pending"; });

// Lambda (C# 3+) -- concise, inline:
orders.Where(o => o.Status == "Pending");
```

Lambdas are what made LINQ practical — passing filtering, projection, and ordering logic inline without cluttering the codebase with named one-use methods.

Lambda syntax

Expression lambda — single expression, no braces, no return keyword:

```
// parameter => expression
o => o.Amount > 100
(x, y) => x + y
() => DateTime.UtcNow
```

Statement lambda — multiple statements, braces, explicit return:

```
// parameter => { statements }
o =>
{
    var tax = o.Amount * 0.18m;
    return o.Amount + tax;
}

(x, y) =>
{
    Console.WriteLine($"Adding {x} and {y}");
}
```

```
    return x + y;
}
```

Type inference — the compiler infers parameter types from context:

```
// Compiler knows o is Order from the List<Order> context:
var pending = orders.Where(o => o.Status == "Pending");

// You can be explicit if needed:
var pending = orders.Where((Order o) => o.Status == "Pending");
```

Closures — capturing outer variables

Lambdas can capture variables from the enclosing scope. The lambda closes over those variables — they remain alive as long as the lambda is referenced:

```
decimal threshold = 100m;    // outer variable

// Lambda captures 'threshold':
var filtered = orders.Where(o => o.Amount > threshold);

threshold = 500m;    // change the captured variable

// Re-evaluate -- uses the NEW value of threshold:
var filtered2 = orders.Where(o => o.Amount > threshold);
// filtered2 uses threshold = 500m, not 100m
```

Variables are captured by reference, not by value. This causes a classic loop closure bug:

```
// Classic closure bug in loops:
var actions = new List<Action>();
for (int i = 0; i < 5; i++)
{
    actions.Add(() => Console.WriteLine(i));    // captures i by reference
}

actions.ForEach(a => a());    // prints: 5 5 5 5 5 (not 0 1 2 3 4!)
// i is 5 after the loop -- all lambdas see the same i

// Fix -- capture the current value into a new variable:
for (int i = 0; i < 5; i++)
{
    int captured = i;    // new variable each iteration
    actions.Add(() => Console.WriteLine(captured));
}
actions.ForEach(a => a());    // prints: 0 1 2 3 4
```

Key Takeaways for 6.4

- Lambda = anonymous function defined inline: parameter => expression
- Expression lambdas for single expressions; statement lambdas { } for multiple statements
- Type inference means you rarely need to specify parameter types
- Closures capture outer variables by reference — changes to the captured variable are visible
- You have already been writing lambdas throughout LINQ — this formalises what they are

Memory aid: A lambda is like a sticky note — a quick anonymous instruction written on the spot and handed to whoever needs it, rather than going through the effort of writing a full named document.

6.1. Delegates (Func, Action, Predicate)

Builds on: 6.4 (lambdas — delegates are what lambdas get assigned to), 1.11 (methods — delegates are typed references to methods), 2.9 (interfaces — delegates are an alternative to interfaces for passing behaviour).

What is a delegate?

A delegate is a type that holds a reference to a method — or a lambda. It is a type-safe function pointer. Where an interface says 'any object implementing this contract,' a delegate says 'any method with this signature.'

```
// Declaring a delegate type:
public delegate bool OrderFilter(Order order);

// Assigning a named method:
bool IsPending(Order o) => o.Status == "Pending";
OrderFilter filter = IsPending;

// Assigning a lambda:
OrderFilter filter = o => o.Status == "Pending";

// Invoking:
bool result = filter(order); // calls whatever method/lambda is assigned
```

Built-in delegate types — Func, Action, Predicate

You almost never declare custom delegate types. .NET provides three generic delegates that cover almost every scenario:

Func<T, TResult> — takes parameters, returns a value:

```
// Func<TResult> -- no params, returns TResult:
Func<DateTime> now = () => DateTime.UtcNow;
DateTime current = now();

// Func<T, TResult> -- one param, returns TResult:
Func<Order, decimal> getAmount = o => o.Amount;
decimal amount = getAmount(order);

// Func<T1, T2, TResult> -- two params, returns TResult:
Func<decimal, decimal, decimal> add = (x, y) => x + y;
decimal sum = add(100m, 200m); // 300

// Up to 16 type parameters supported

// In LINQ -- Select takes Func<T, TResult>:
orders.Select(o => o.Amount) // Func<Order, decimal>
```

Action<T> — takes parameters, returns void:

```
// Action -- no params, no return:
Action greet = () => Console.WriteLine("Hello");
greet();    // "Hello"

// Action<T> -- one param, no return:
Action<Order> log = o => Console.WriteLine($"Order {o.Id}: {o.Amount:C}");
log(order);

// Action<T1, T2> -- two params, no return:
Action<string, int> print = (msg, count) =>
{
    for (int i = 0; i < count; i++)
        Console.WriteLine(msg);
};
print("Hello", 3);    // prints Hello three times

// In List<T>.ForEach -- takes Action<T>:
orders.ForEach(o => Console.WriteLine(o.Id));
```

Predicate<T> — takes one parameter, returns bool:

```
// Predicate<T> -- equivalent to Func<T, bool>:
Predicate<Order> isPending = o => o.Status == "Pending";
bool result = isPending(order);

// In List<T>.Find -- takes Predicate<T>:
Order? found = orders.Find(o => o.Status == "Pending");

// Predicate<T> and Func<T, bool> have the same signature but are NOT the same type:
Func<Order, bool> funcFilter = o => o.Status == "Pending";
// orders.Find(funcFilter);    // compile error -- Find wants Predicate<T>

// In new code, prefer Func<T, bool> over Predicate<T> -- more composable with LINQ
```

Delegates as parameters — passing behaviour

```
// Method that accepts a delegate as parameter:
public List<Order> FilterOrders(List<Order> orders, Func<Order, bool> filter)
{
    return orders.Where(filter).ToList();
}

// Callers pass different behaviour:
var pending    = FilterOrders(orders, o => o.Status == "Pending");
var highValue  = FilterOrders(orders, o => o.Amount > 1000);
var recentLarge = FilterOrders(orders,
    o => o.CreatedAt > DateTime.UtcNow.AddDays(-7) && o.Amount > 500);

// Processing pipeline with multiple delegate parameters:
public void ProcessOrders(
    List<Order> orders,
    Func<Order, bool> filter,
    Action<Order> process,
    Func<Order, string> getReport)
```

```

{
    foreach (var order in orders.Where(filter))
    {
        process(order);
        Console.WriteLine(getReport(order));
    }
}

```

```

// Wire up with lambdas:
ProcessOrders(
    orders,
    filter:    o => o.Status == "Pending",
    process:   o => o.Status = "Processing",
    getReport: o => $"Processed order {o.Id}"
);

```

Delegates stored in fields — pluggable behaviour

```

public class OrderValidator
{
    private readonly Func<Order, bool> _isValid;
    private readonly Action<string> _onError;

    public OrderValidator(Func<Order, bool> isValid, Action<string> onError)
    {
        _isValid = isValid;
        _onError = onError;
    }

    public bool Validate(Order order)
    {
        if (!_isValid(order))
        {
            _onError($"Order {order.Id} failed validation");
            return false;
        }
        return true;
    }
}

// Inject behaviour at construction:
var validator = new OrderValidator(
    isValid: o => o.Amount > 0 && !string.IsNullOrEmpty(o.CustomerName),
    onError: msg => _logger.LogWarning(msg)
);

```

Key Takeaways for 6.1

- A delegate is a type-safe reference to a method or lambda
- Func — has parameters, returns a value
- Action — has parameters, returns void
- Predicate — takes one parameter, returns bool (used by older APIs like List.Find)
- Prefer Func over Predicate in new code — more composable with LINQ

- Delegates enable passing behaviour as a parameter — the foundation of callbacks, strategies, and LINQ

Delegate type	Signature	Common use
Func<TResult>	() => TResult	Factory, provider
Func<T, TResult>	T => TResult	Selector, transformer
Func<T, bool>	T => bool	Filter, condition check
Action	() => void	Callback, side effect
Action<T>	T => void	Notification, logging
Predicate<T>	T => bool	Older BCL APIs

Memory aid: A delegate is a labelled envelope — it describes what kind of letter fits inside (method signature). A lambda is the letter itself. You can swap out the letter as long as it fits the envelope.

Debugging Tips

Symptom	Cause	Fix
Delegate invocation throws NullReferenceException	Delegate variable is null -- no method assigned	Check for null before invoking: myDelegate?.Invoke() or if (myDelegate != null)
Cannot pass Func where Predicate expected	Func and Predicate are different types despite same signature	Use a lambda: list.Find(o => myFunc(o)) to bridge the type difference
Delegate captures stale value	Closure captures variable by reference -- value changed after capture	Copy the value into a local variable before the lambda if you want the value at capture time
Lambda with statement body gives compile error in LINQ expression tree	EF Core expression trees require expression lambdas, not statement lambdas	Keep EF Core lambda bodies as single expressions; move complex logic to in-memory

6.2. Multicast Delegates

Builds on: 6.1 (delegates — multicast is a feature of all delegates), 6.3 (events — multicast delegates are how events work internally).

What is a multicast delegate?

A regular delegate holds one method. A multicast delegate holds multiple methods — all called in sequence when the delegate is invoked. Every delegate in C# is multicast by default.

```

Action<string> log = msg => Console.WriteLine($"Console: {msg}");
Action<string> save = msg => File.AppendAllText("log.txt", msg + "\n");
Action<string> email = msg => SendEmail("admin@zoho.com", msg);

// Combine with + operator:
Action<string> all = log + save + email;

// Invoking 'all' calls all three in sequence:
all("Order processed");
// Console: Order processed

```

```
// (saves to file)
// (sends email)
```

Adding and removing

```
Action<string> notifier = msg => Console.WriteLine(msg);
notifier += msg => File.AppendAllText("log.txt", msg);    // add

// To remove, keep a reference -- lambda instances are different objects:
Action<string> consolePart = msg => Console.WriteLine(msg);
notifier += consolePart;
notifier -= consolePart;    // works -- same reference

// This does NOT work -- different lambda instance:
notifier -= msg => Console.WriteLine(msg);    // no-op, silently ignored
```

Return values in multicast delegates

```
// Only the LAST invoked method's return value is kept:
Func<int> counter = () => 1;
counter += () => 2;
counter += () => 3;

int result = counter();    // result = 3 (only last return value)
// 1 and 2 are discarded

// This is why multicast delegates with return values are rarely used.
// Multicast is most natural with Action (void return).
```

Safe invocation and inspecting the invocation list

```
// Safe invocation -- null if no methods attached:
notifier?.Invoke("Hello");

// Inspect all attached methods:
var methods = notifier.GetInvocationList();
Console.WriteLine($"Attached handlers: {methods.Length}");

// Manually invoke each and collect results (avoids last-only limitation):
Func<int> multiFunc = () => 1;
multiFunc += () => 2;
multiFunc += () => 3;

var results = multiFunc.GetInvocationList()
    .Cast<Func<int>>()
    .Select(f => f())
    .ToList();
// results = [1, 2, 3] -- all return values collected
```

Key Takeaways for 6.2

- Every delegate in C# is multicast -- it can hold multiple methods
- + adds a method to the delegate; - removes it
- On invocation, all attached methods are called in order

- Only the last return value is kept -- use Action (void) for multicast scenarios
- Use ?.Invoke() to safely call a possibly-null delegate
- Use GetInvocationList() to manually invoke each handler and collect all results

Memory aid: A multicast delegate is like a group email. You send one message and it goes to everyone on the list. You can add or remove people. But if you ask for a reply, you only get the last person's answer.

Debugging Tips

Symptom	Cause	Fix
Removing a lambda from a multicast delegate has no effect	Lambda instances are different objects -- -= compares by reference	Store the lambda in a variable before adding, then use that variable to remove
Multicast delegate only returns last value	By design -- only the final invoked method's return value is kept	Use GetInvocationList() and invoke each handler manually to collect all results
Exception in one handler stops others from running	An unhandled exception in a multicast handler aborts the remaining invocations	Wrap each handler invocation in try/catch, or use GetInvocationList() to invoke individually
Delegate is null despite += being called	The += was called on a local copy, not the field	Ensure += is called on the field directly, not a local variable copy

6.3. Events (event keyword, publisher/subscriber pattern)

Builds on: 6.1 (delegates — events are built on delegates), 6.2 (multicast delegates — events use them internally), 2.9 (interfaces — events define contracts between objects).

What are events and why do they exist?

Multicast delegates allow multiple subscribers — but they have a safety problem: any code holding the delegate can invoke it or clear all subscribers by assigning null. Events add restrictions that make the publisher/subscriber pattern safe:

```
// With a plain delegate field -- UNSAFE:
public Action<OrderEventArgs> OrderPlaced;    // public field

// External code can:
service.OrderPlaced = null;                  // clears ALL subscribers
service.OrderPlaced(args);                   // invokes from outside the class

// With event -- SAFE:
public event Action<OrderEventArgs> OrderPlaced;

// External code can ONLY:
service.OrderPlaced += handler;              // subscribe
service.OrderPlaced -= handler;              // unsubscribe
// service.OrderPlaced = null;               // compile error
// service.OrderPlaced(args);                // compile error
// Only the declaring class can invoke or clear
```

The standard event pattern with EventHandler

```
// Event args -- carries data about what happened:
public class OrderPlacedEventArgs : EventArgs
{
    public int OrderId { get; }
    public string CustomerEmail { get; }
    public decimal Amount { get; }

    public OrderPlacedEventArgs(int orderId, string customerEmail, decimal amount)
    {
        OrderId = orderId;
        CustomerEmail = customerEmail;
        Amount = amount;
    }
}

// Publisher -- raises the event:
public class OrderService
{
    // EventHandler<TEventArgs> is the .NET standard event signature:
    public event EventHandler<OrderPlacedEventArgs>? OrderPlaced;
    public event EventHandler<OrderPlacedEventArgs>? OrderCancelled;

    private void OnOrderPlaced(OrderPlacedEventArgs args)
    {
        // Null check -- no subscribers might be attached:
        OrderPlaced?.Invoke(this, args);
    }

    public async Task<Order> CreateOrderAsync(CreateOrderDto dto)
    {
        var order = new Order(dto);
        await _repo.SaveAsync(order);

        OnOrderPlaced(
            new OrderPlacedEventArgs(order.Id, order.CustomerEmail, order.Amount));

        return order;
    }
}

// Subscribers -- react to the event:
public class EmailNotificationService
{
    private readonly OrderService _orderService;

    public EmailNotificationService(OrderService orderService)
    {
        _orderService = orderService;
        _orderService.OrderPlaced += OnOrderPlaced;
        _orderService.OrderCancelled += OnOrderCancelled;
    }

    private async void OnOrderPlaced(object? sender, OrderPlacedEventArgs e)
    {

```

```

        await SendConfirmationEmailAsync(e.CustomerEmail, e.OrderId, e.Amount);
    }

    private async void OnOrderCancelled(object? sender, OrderPlacedEventArgs e)
    {
        await SendCancellationEmailAsync(e.CustomerEmail, e.OrderId);
    }

    // Always unsubscribe when done to prevent memory leaks:
    public void Unsubscribe()
    {
        _orderService.OrderPlaced -= OnOrderPlaced;
        _orderService.OrderCancelled -= OnOrderCancelled;
    }
}

```

Simpler events with Action

For internal/domain use, the full EventHandler ceremony is not always needed:

```

public class OrderProcessor
{
    // Simple events using Action:
    public event Action<int>? OrderCompleted;
    public event Action<int, string>? OrderFailed;

    public async Task ProcessAsync(int orderId)
    {
        try
        {
            await DoProcessingAsync(orderId);
            OrderCompleted?.Invoke(orderId);
        }
        catch (Exception ex)
        {
            OrderFailed?.Invoke(orderId, ex.Message);
        }
    }
}

// Subscribe with lambdas:
var processor = new OrderProcessor();
processor.OrderCompleted += id => Console.WriteLine($"Order {id} done");
processor.OrderFailed += (id, msg) => Console.WriteLine($"Order {id} failed: {msg}");

```

Memory leak warning — always unsubscribe

The most common event bug: subscribing but never unsubscribing causes memory leaks because the publisher holds a reference to the subscriber, preventing garbage collection:

```

// LEAK -- subscriber never unsubscribes:
public class OrderDashboard
{
    public OrderDashboard(OrderService service)
    {
        service.OrderPlaced += OnOrderPlaced;    // service holds reference to this
    }
}

```

```

        // When dashboard is "disposed", service still holds reference -- not GC'd
    }

    private void OnOrderPlaced(object? sender, OrderPlacedEventArgs e) { ... }
}

// FIX -- implement IDisposable and unsubscribe:
public class OrderDashboard : IDisposable
{
    private readonly OrderService _service;

    public OrderDashboard(OrderService service)
    {
        _service = service;
        _service.OrderPlaced += OnOrderPlaced;
    }

    private void OnOrderPlaced(object? sender, OrderPlacedEventArgs e) { ... }

    public void Dispose()
    {
        _service.OrderPlaced -= OnOrderPlaced;    // unsubscribe
    }
}

```

Events vs delegates vs interfaces

Events:

Use when multiple listeners react to something that happened.
 Publisher does not know or care who listens.
 Examples: order placed notification, UI button click, logging hooks.

Delegates (Func/Action as parameters):

Use when passing a single behaviour into a method.
 The method needs exactly one function to call.
 Examples: LINQ predicates, strategy callbacks, factory methods.

Interfaces:

Use when multiple methods define a capability contract.
 Dependency injection, polymorphism, testability.
 Examples: IOrderRepository, IEmailService, IValidator<T>.

Realistic scenario — putting it all together

```

// Delegates for strategy pattern + events for audit:
public class PricingEngine
{
    private readonly Func<Order, decimal> _discountStrategy;
    private readonly Func<decimal, decimal> _taxCalculator;

    public event Action<Order, decimal>? PriceCalculated;

    public PricingEngine(
        Func<Order, decimal> discountStrategy,
        Func<decimal, decimal> taxCalculator)
    {
    }
}

```

```

    {
        _discountStrategy = discountStrategy;
        _taxCalculator     = taxCalculator;
    }

    public decimal CalculateAndNotify(Order order)
    {
        var discount      = _discountStrategy(order);
        var afterDiscount = order.Amount - discount;
        var tax           = _taxCalculator(afterDiscount);
        var total         = afterDiscount + tax;

        PriceCalculated?.Invoke(order, total);
        return total;
    }
}

// Wire up:
var engine = new PricingEngine(
    discountStrategy: o => o.Amount > 1000 ? o.Amount * 0.1m : 0,
    taxCalculator:    amount => amount * 0.18m
);

engine.PriceCalculated += (order, total) =>
    _logger.LogInformation("Order {Id} priced at {Total:C}", order.Id, total);

engine.PriceCalculated += (order, total) =>
    _auditService.RecordAsync(order.Id, total);

decimal price = engine.CalculateAndNotify(order);

```

Key Takeaways for 6.3

- Events restrict delegate access -- external code can only subscribe/unsubscribe, not invoke or clear
- EventHandler is the .NET standard event signature -- sender + event args
- OrderPlaced?.Invoke(this, args) -- null-safe invocation when no subscribers are attached
- Always unsubscribe from events when the subscriber is no longer needed -- memory leak prevention
- Use event Action for simple internal events; EventHandler for public APIs
- Events = publisher does not know subscribers; delegates = explicitly pass one callback

Memory aid: An event is a locked letterbox — anyone can drop a letter in (subscribe) but only the owner of the box can open it and read the letters (invoke). A delegate is an unlocked box — anyone can put in or take out letters.

Debugging Tips

Symptom	Cause	Fix
Event handler never called	Forgot to subscribe with +=, or subscribing code never runs	Add the subscription; set a breakpoint on the += line to confirm it executes

Symptom	Cause	Fix
Event fires but causes <code>NullReferenceException</code>	No null check before invoke -- event has no subscribers	Use <code>?.Invoke()</code> not <code>.Invoke()</code>
Memory leak -- objects not garbage collected	Subscribed to event but never unsubscribed	Implement <code>IDisposable</code> and unsubscribe in <code>Dispose()</code>
Cannot invoke event from outside the class	event keyword prevents external invocation -- correct behaviour	Add a public method on the publisher to raise it if needed externally
async void event handler exceptions are swallowed	async void exceptions cannot be caught by the caller	Wrap the body in try/catch and handle exceptions inside the handler

Section 6 — Complete Summary

All topics at a glance

Concept	What it is	When to use
-----+-----+-----		
Lambda	Anonymous inline function	Everywhere -- LINQ, callbacks
<code>Func<T, TResult></code>	Delegate that returns a value	Strategy, selector, factory
<code>Action<T></code>	Delegate that returns void	Callback, notification
<code>Predicate<T></code>	<code>Func<T, bool></code> specialisation	Older APIs (<code>List.Find</code>)
Multicast delegate	Delegate holding multiple methods	Events, notification chains
event	Restricted multicast delegate	Publisher/subscriber pattern

.NET-specific rules:

- `Func`, `Action`, `Predicate` -- no custom delegate declarations needed in modern C#
- event keyword prevents external invocation and clearing -- always use for public event fields
- `?.Invoke()` -- always null-check before invoking an event
- Unsubscribing from events -- mandatory when subscriber lifetime is shorter than publisher
- `async void` is acceptable ONLY for event handlers -- never elsewhere in code

Section 6 complete. All 4 roadmap topics (6.1 through 6.4) covered. Topics taught in order: 6.4 (lambda expressions -- foundation), 6.1 (delegates -- `Func`, `Action`, `Predicate`), 6.2 (multicast delegates), 6.3 (events -- publisher/subscriber pattern). Say next to begin Section 7: Asynchronous Programming.